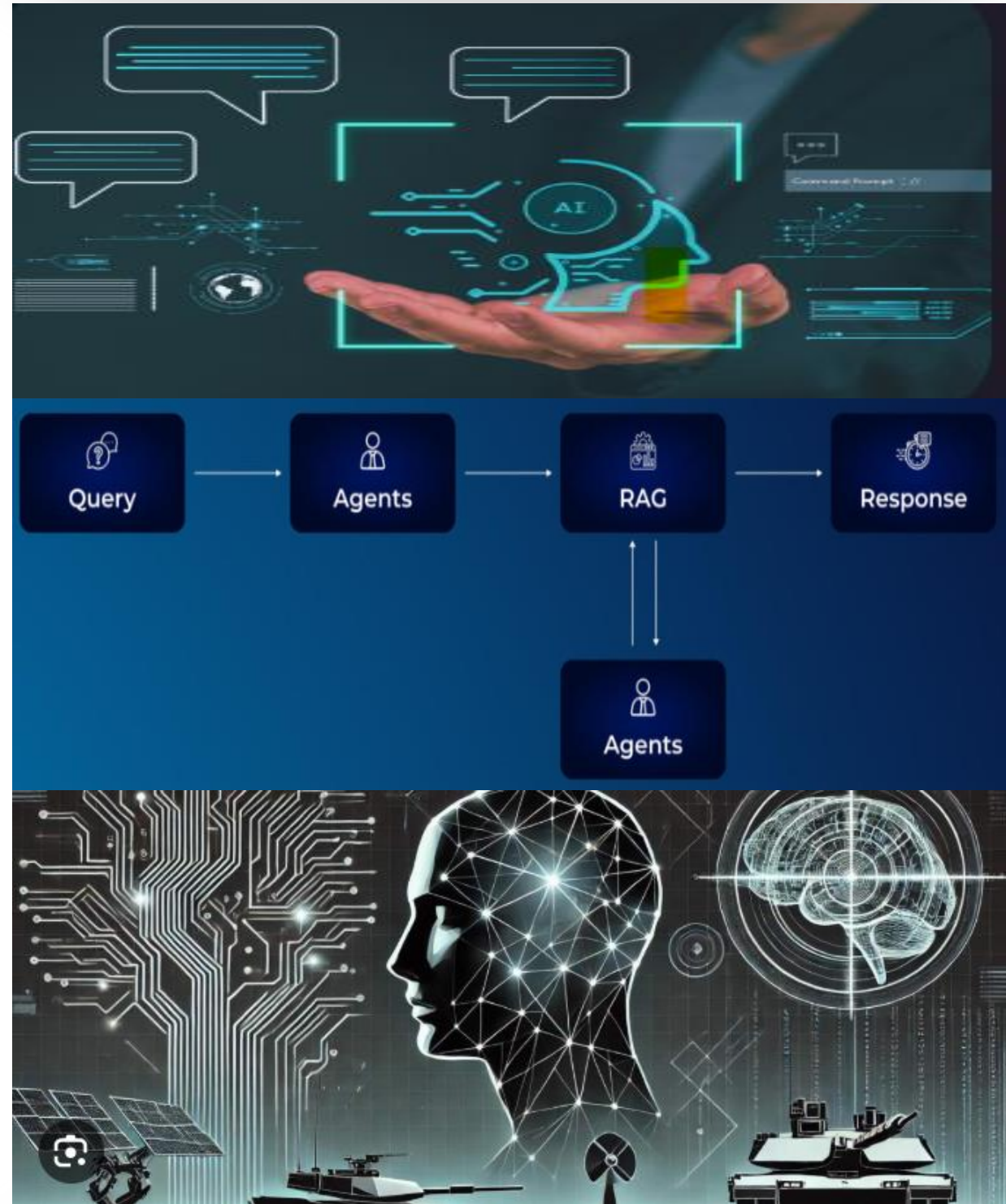




Top Agentic RAG Architecture

(A Comprehensive Guide)

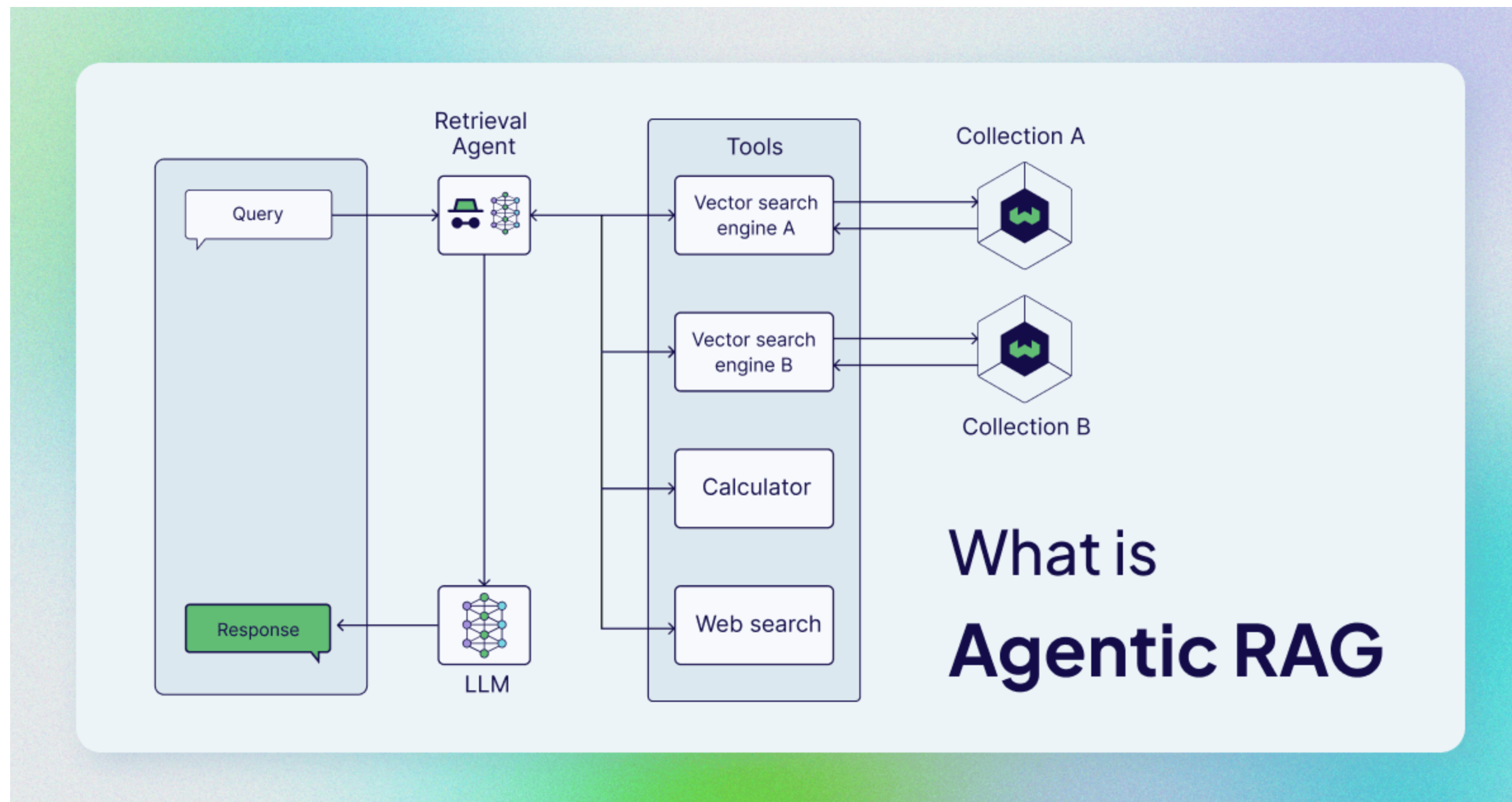


Karn Singh



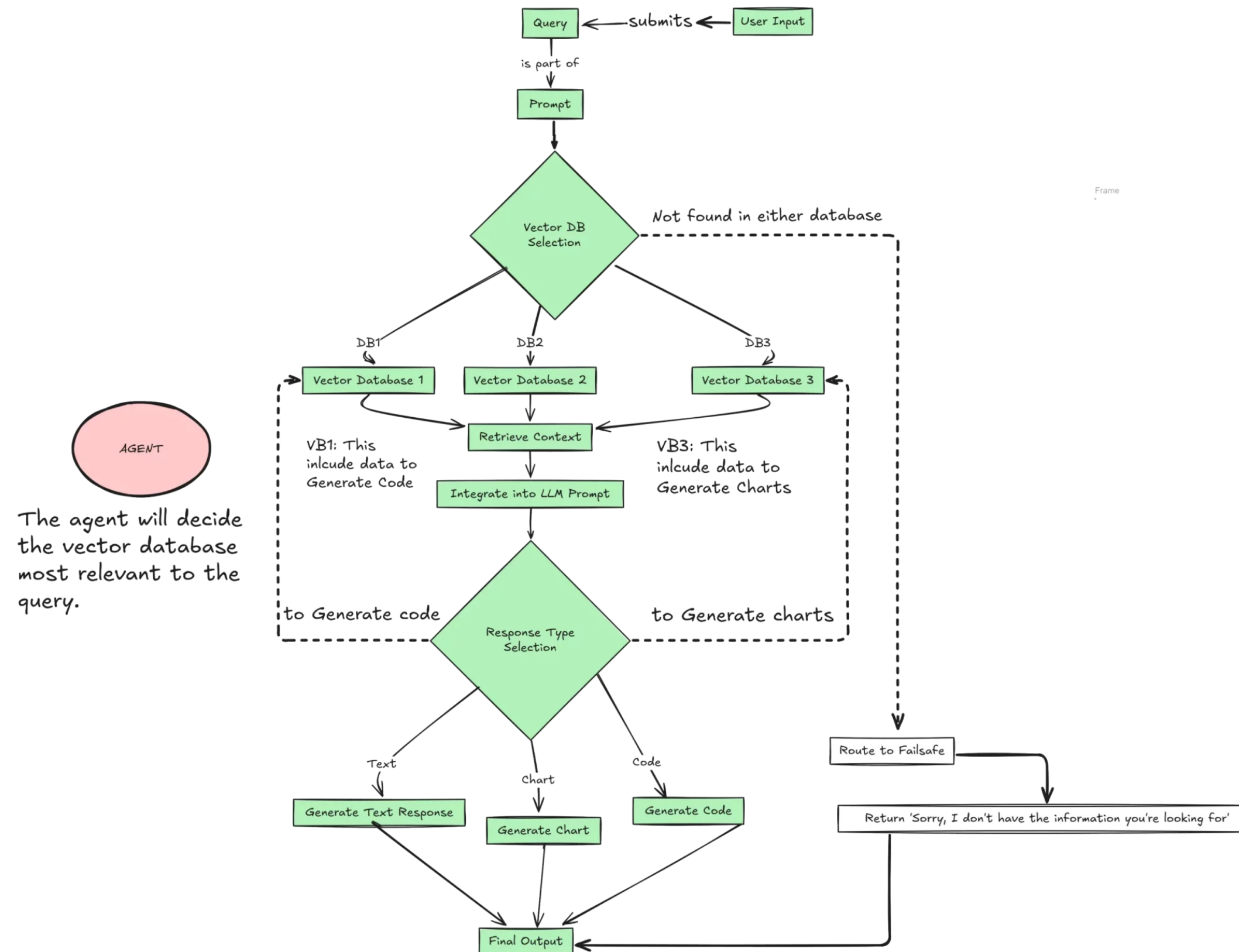
What is **Agentic RAG** : Merging RAG with AI Agents

Agentic RAG is an advanced implementation of Retrieval-Augmented Generation (RAG) that integrates AI agents to enhance traditional RAG systems.



Architecture of Agentic RAG

Combining the AI Agents and RAG workflow, here's the architecture of Agentic RAG:



How **Agentic RAG** works

Agentic RAG combines the structured retrieval and knowledge integration capabilities of RAG with the autonomy and adaptability of AI agents. Here's how it works:

Dynamic Knowledge Retrieval

Agents can retrieve specific, up-to-date information on the fly, ensuring relevance and accuracy.

Intelligent Decision-Making:

Agents apply advanced reasoning to process data, generating solutions and answering questions with depth.

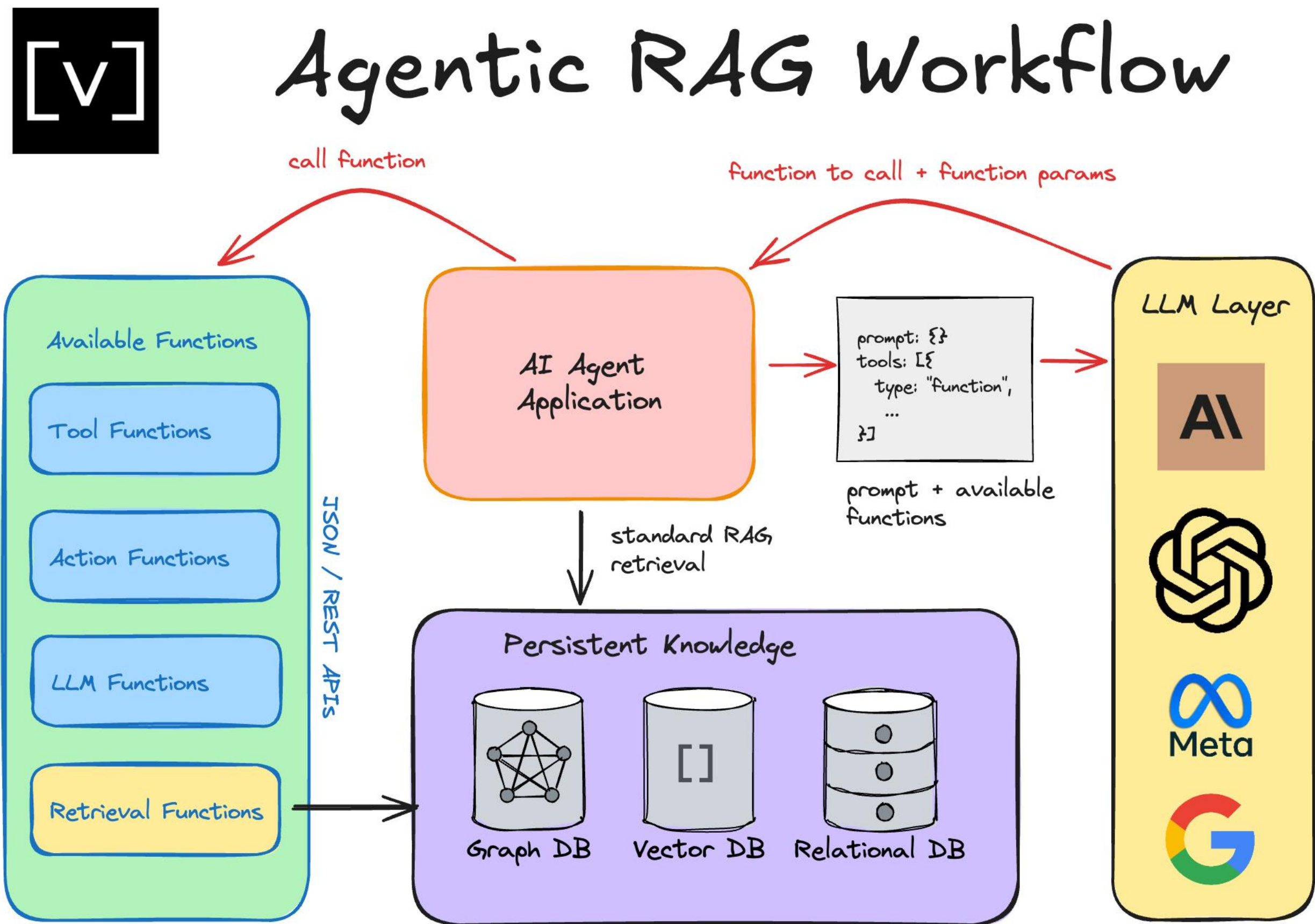
Task-Oriented Execution

Capable of executing multi-step tasks and adapting to changing objectives based on feedback.

Continuous Improvement:

Agents learn over time, enhancing their retrieval strategies and task execution for greater efficiency and effectiveness.

Agentic RAG Workflow



Key Features:

Agentic RAG transforms the traditional RAG approach by introducing intelligent agents that enhance decision-making and retrieval processes.

Key Features :

AI Agents Unlike conventional RAG, which relies mainly on large language models (LLMs), Agentic RAG incorporates autonomous agents capable of performing complex tasks beyond simple query handling.	Dynamic Query Handling The architecture allows for dynamic adjustments based on user queries, enabling it to manage complex and nuanced questions effectively.	Enhanced Problem-Solving Agents act like skilled researchers, navigating multiple documents, comparing information, generating summaries, and synthesizing responses.	Tool Utilization It supports integration with various external tools and resources, such as databases and APIs, to improve information retrieval.	Continuous Learning Agentic RAG systems learn from interactions over time, adapting their strategies based on feedback and new data.	Improved Accuracy By combining retrieval with generation, Agentic RAG reduces errors and enhances the reliability of responses.
--	--	---	---	--	---

7 Agentic RAG :



Agentic RAG Router:

Definition: Agentic RAG combines structured retrieval and knowledge integration with the autonomy and adaptability of AI agents.

Dynamic Query Routing: Systems are designed to route user queries to appropriate tools or data sources for enhanced LLM capabilities.

Retrieval Mechanisms: Integrates various retrieval methods (e.g., database search, web search) with generative strengths of LLMs.

Bridging Knowledge Gaps: Addresses the limitations of static LLMs by enabling dynamic retrieval from live or domain-specific data sources.

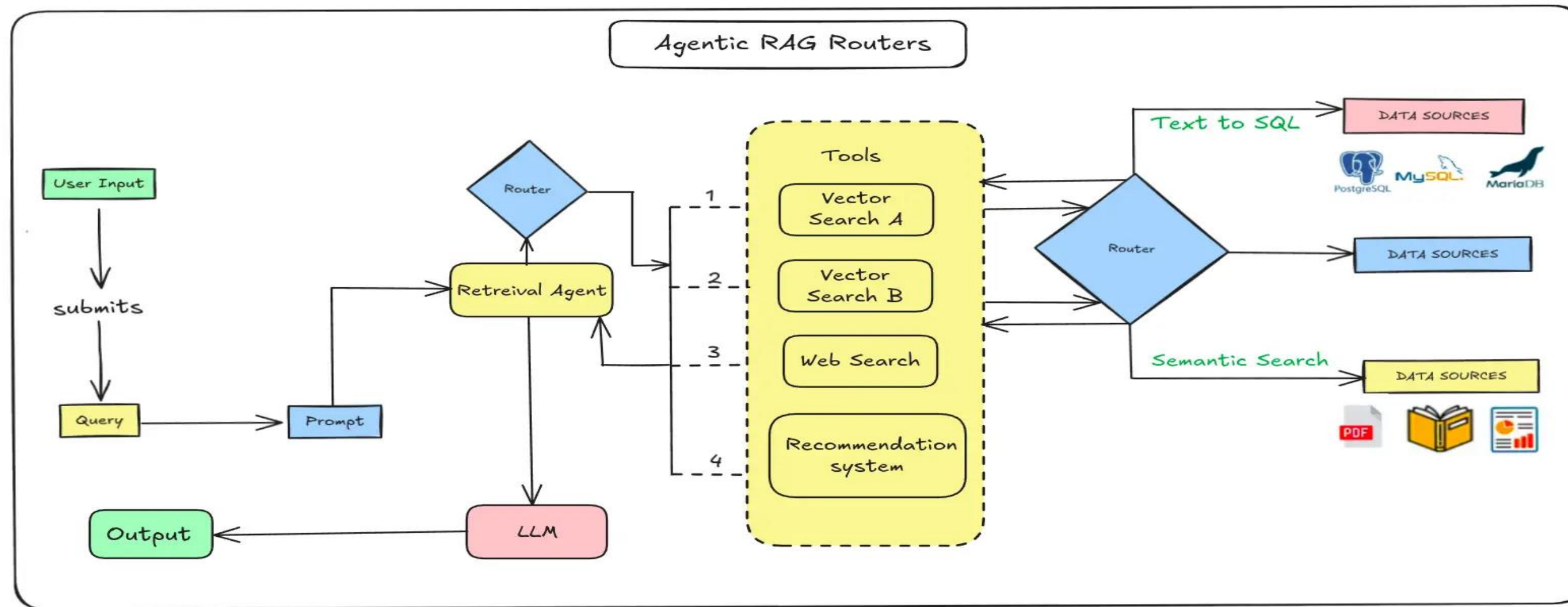
Applications: Question answering , Data analysis, Real-time information retrieval, Recommendation generation

Intelligent Decision-Making: Agents reason and decide which tools to use for retrieving and processing data effectively.

Task-Oriented Execution: Capable of executing multi-step tasks and adjusting strategies based on feedback.

Continuous Improvement: Agents learn over time, enhancing their retrieval strategies and overall efficiency.

Agentic RAG Router Architecture:



The architecture shown in the diagram provides a detailed visualization of how Agentic RAG Routers operate. Let's break down the components and flow:

1. User Input and Query Processing

1. **User Input:** A user submits a query, which is the entry point for the system. This could be a question, a command, or a request for specific data.
2. **Query:** The user input is parsed and formatted into a query, which the system can interpret.

2. Retrieval Agent

1. The **Retrieval Agent** serves as the core processing unit. It acts as a coordinator, deciding how to handle the query. It evaluates:
 1. The intent of the query.
 2. The type of information required (structured, unstructured, real-time, recommendations).

Agentic RAG Router Architecture:

3. Router

A **Router** determines the appropriate tool(s) to handle the query:

Vector Search: Retrieves relevant documents or data using semantic embeddings.

Web Search: Accesses live information from the internet.

Recommendation System: Suggests content or results based on prior user interactions or contextual relevance.

Text-to-SQL: Converts natural language queries into SQL commands for accessing structured databases.

4. Tools: The tools listed here are modular and specialized:

Vector Search A & B: Designed to search semantic embeddings for matching content in vectorized forms, ideal for unstructured data like documents, PDFs, or books.

Web Search: Accesses external, real-time web data.

Recommendation System: Leverages AI models to provide user-specific suggestions.

5. Data Sources: The system connects to diverse data sources:

Structured Databases: For well-organized information (e.g., SQL-based systems).

Unstructured Sources: PDFs, books, research papers, etc.

External Repositories: For semantic search, recommendations, and real-time web queries.

6. LLM Integration: Once data is retrieved, it is fed into the LLM:

The LLM synthesizes the retrieved information with its generative capabilities to create a coherent, human-readable response.

7. Output: The final response is sent back to the user in a clear and actionable format.

Query Planning Agentic RAG :

Purpose: Designed to efficiently handle complex queries by leveraging multiple parallelizable subqueries across diverse data sources.

Key Features:

- **Intelligent Query Division:** Breaks down complex queries into smaller, manageable subqueries for efficient processing.
- **Distributed Processing:** Executes subqueries across various data sources simultaneously, enhancing response speed & accuracy.
- **Response Synthesis:** Combines results from subqueries to deliver comprehensive and coherent answers to the user.

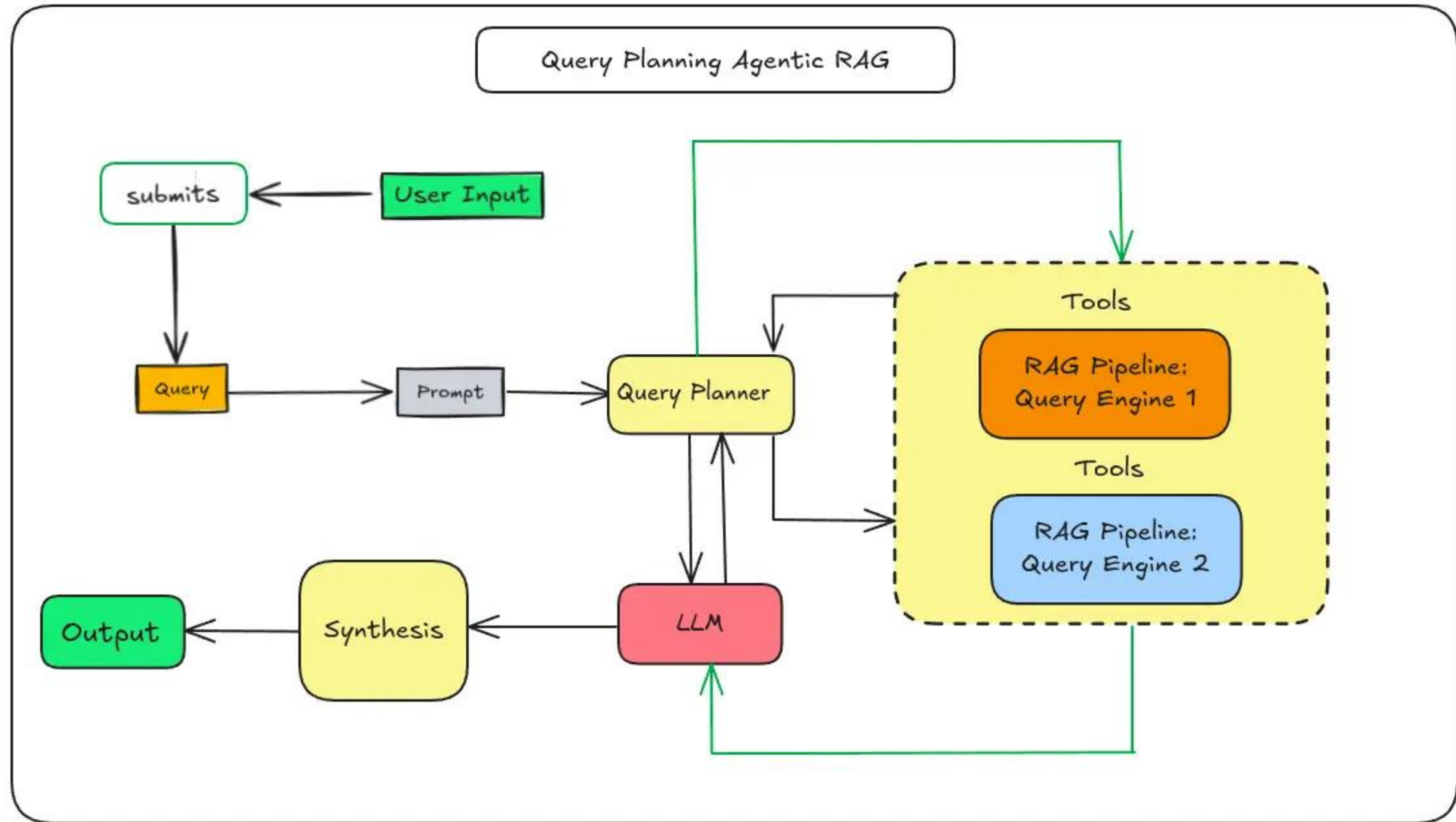
Methodology:

- **Adaptive Reasoning:** Utilizes a reasoner to interpret user intent and strategize information retrieval.
- **Collaborative Agent Network:** Employs a network of specialized agents, each with distinct skills, to manage extensive datasets effectively.
- **Dynamic Planning and Execution:** Adapts in real-time to evolving queries and information landscapes, optimizing the retrieval process.

Applications:

- Effective in scenarios requiring detailed question answering, data analysis, real-time information retrieval, and recommendation generation.

Query Planning Agentic RAG :



Query Planning Agentic RAG :

Core Components

- **User Input and Query Submission:** User submits a query, which is processed for further handling.
- **Query Planner:**
 - Central component that interprets the query.
 - Generates prompts for downstream components.
 - Decides which tools (query engines) to invoke.
- **Tools:**
 - Specialized pipelines (e.g., RAG pipelines) with query engines for retrieving information from external sources (databases, documents, APIs).
 - Retrieved information is sent back to the Query Planner.
- **LLM (Large Language Model):**
 - Acts as the synthesis engine for reasoning and response generation.
 - Interacts with the Query Planner, receiving prompts and providing context-aware responses.
- **Synthesis and Output:**
 - Combines retrieved information and LLM responses into a coherent answer.
 - Presents the final synthesized result to the user.

Adaptive RAG :

Purpose: Designed to efficiently handle complex queries by leveraging multiple parallelizable subqueries across diverse data sources.

Key Features:

Intelligent Query Division: Breaks down complex queries into smaller, manageable subqueries for efficient processing.

Distributed Processing: Executes subqueries across various data sources simultaneously, enhancing response speed and accuracy.

Response Synthesis: Combines results from subqueries to deliver comprehensive and coherent answers to the user.

Methodology:

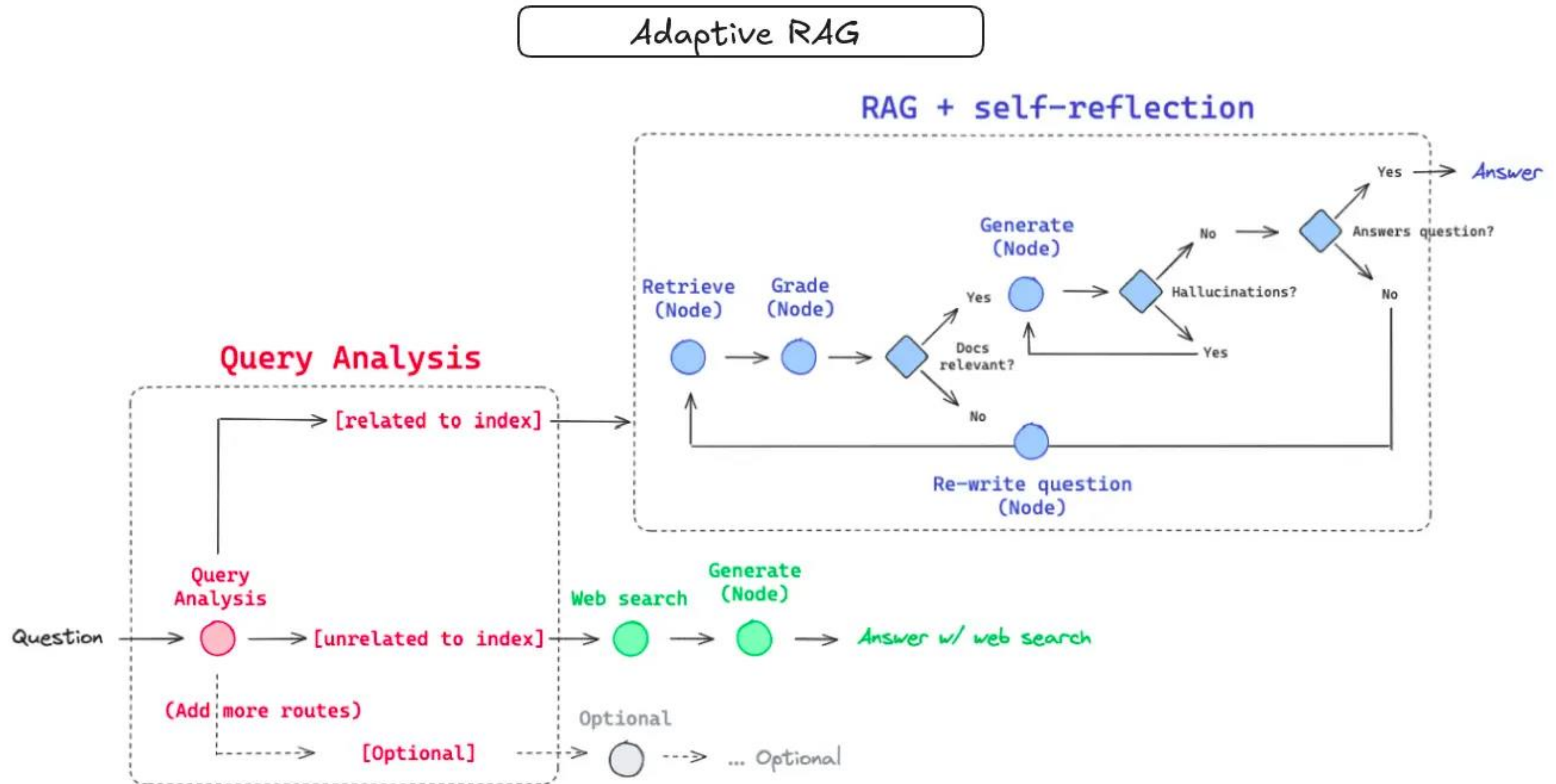
Adaptive Reasoning: Utilizes a reasoner to interpret user intent and strategize information retrieval.

Collaborative Agent Network: Employs a network of specialized agents, each with distinct skills, to manage extensive datasets effectively.

Dynamic Planning and Execution: Adapts in real-time to evolving queries and information landscapes, optimizing the retrieval process.

Applications: Effective in scenarios requiring detailed question answering, data analysis, real-time information retrieval, and recommendation generation.

Adaptive RAG :



Adaptive RAG :

Process Steps:

- **Query Analysis:** Analyzes user queries to determine the best retrieval pathway.
 - **Route Determination:** Classifies queries as related or unrelated to the indexed content.
 - **Optional Routes:** Includes pathways for domain-specific tools or APIs.
- **RAG + Self-Reflection:**
 - **Retrieve Node:** Gathers documents from the indexed database.
 - **Grade Node:** Assesses relevance of retrieved documents.
 - **Generate Node:** Produces a response based on relevant documents.
 - **Self-Reflection Steps:** Iteratively checks if the answer meets the query requirements and adjusts if necessary.
 - **Hallucinations Check:** Detects inaccuracies and refines queries for better retrieval.
- **Web Search for Unrelated Queries:**

If unrelated, performs a web search and generates a response based on retrieved data.
- **Outcome:**

Delivers accurate, contextually relevant responses while improving computational efficiency through tailored query strategies.

Corrective RAG :

Purpose: Enhances retrieval accuracy and generation quality by self-correcting results.

Key Features:

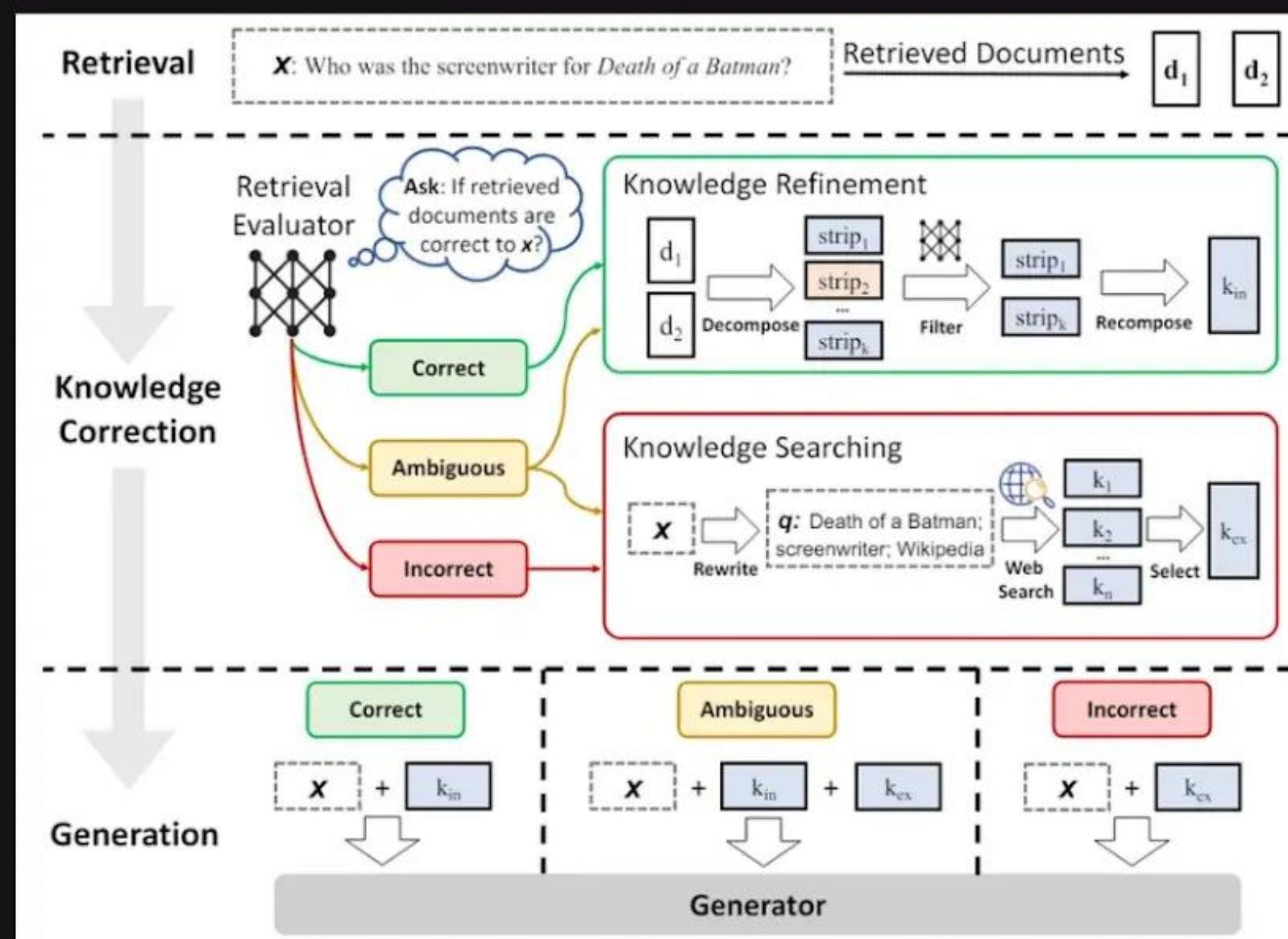
- Retrieval Evaluator: Assesses relevance and reliability of retrieved documents, assigning a confidence score.
- Triggered Actions: Based on confidence scores, actions are categorized as Correct, Ambiguous, or Incorrect.
- Web Searches for Augmentation: Integrates large-scale web searches to improve retrieval results beyond static databases.
- Decompose-Then-Recompose Algorithm: Extracts key information from documents while filtering out irrelevant content.
- Plug-and-Play Capability: Easily integrates with existing RAG systems without extensive modifications.

Benefits:

Improves robustness of generation, enhances document utilization, and adapts dynamically to retrieval inaccuracies.

Corrective RAG :

Corrective RAG Workflow proposed in the CRAG paper



- **Step 1:**
 - Retrieve context documents from vector database from the input query
- **Step 2:**
 - Use an LLM to check if retrieved documents are relevant to input question
- **Step 3:**
 - If all documents are relevant (Correct), no specific action needed
- **Step 4:**
 - If some or all documents are not relevant (Ambiguous OR Incorrect), rephrase the query and search the web to get relevant context information
- **Step 5:**
 - Send rephrased query and context documents or information to the LLM for response generation

Self Reflective RAG :

Definition: Self-Reflective Retrieval-Augmented Generation (SELF-RAG) enhances the quality and factuality of large language models (LLMs) through on-demand retrieval and self-reflection.

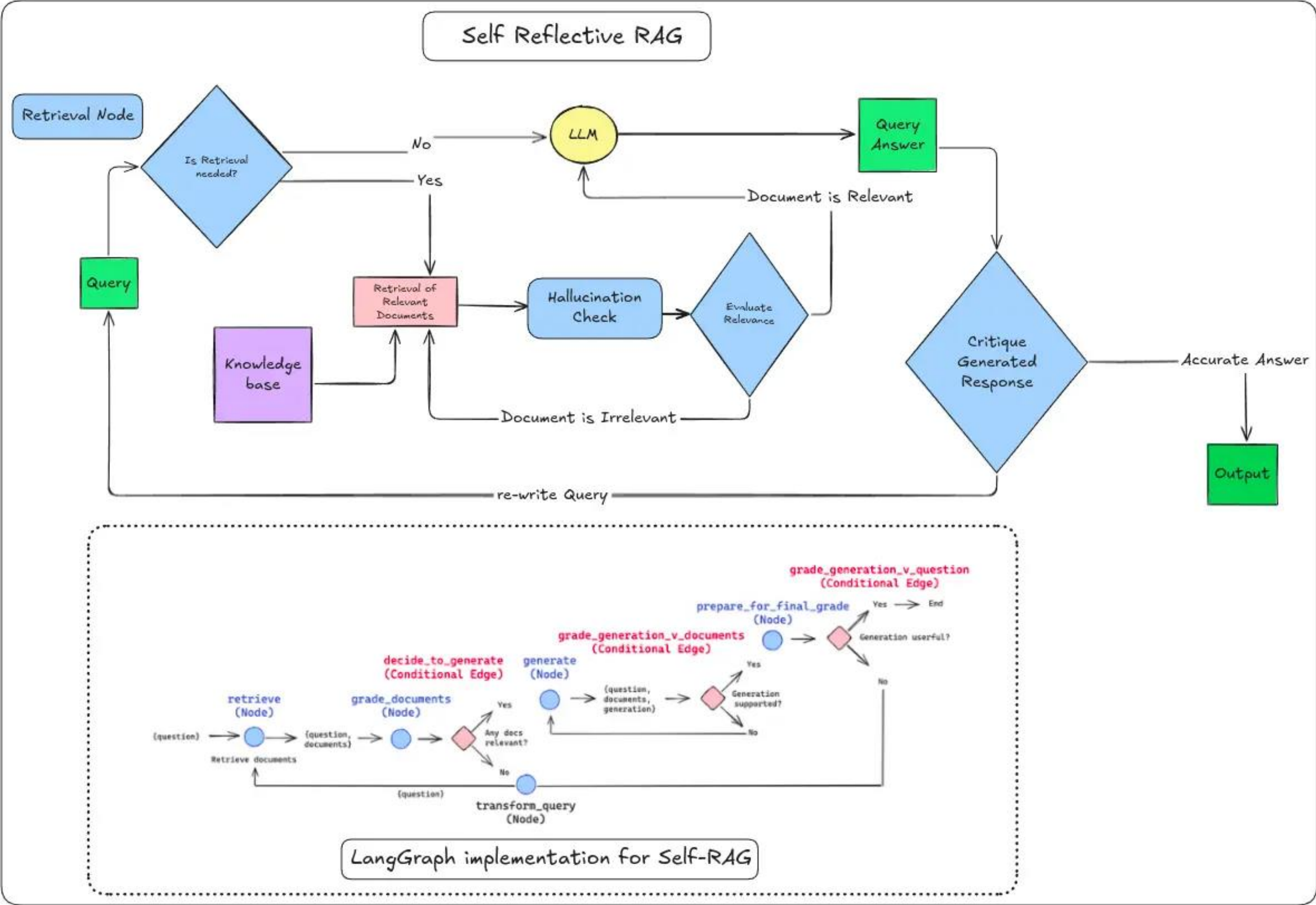
Key Mechanisms:

- **Reflection Tokens:** Special tokens that indicate the need for retrieval or assess the quality of generated outputs.
- **On-Demand Retrieval:** Determines if additional information is needed to augment the generation process, retrieving relevant passages as necessary.
- **Self-Critique Process:** Evaluates the generated content against retrieved passages to ensure accuracy and relevance.

Benefits:

- **Improved Factuality:** Reduces inaccuracies (hallucinations) in generated responses by ensuring they are supported by retrieved data.
- **Enhanced Versatility:** Maintains the LLM's creative capabilities while improving response quality.
- **Dynamic Adaptation:** Adapts to varying task requirements through customizable reflection mechanisms.

Self Reflective RAG :



Self Reflective RAG Process Overview :

- **Query Submission:** User submits a query.
- **First Decision Point:**
 - Is Retrieval Needed?
 - NO: Query goes directly to the LLM for processing.
 - YES: Proceed to retrieval steps.
- **Knowledge Base Integration:** Connects to the retrieval of relevant documents.
- **Relevance Evaluation:**
 - Retrieved documents are evaluated as "Relevant" or "Irrelevant."
 - **Irrelevant documents trigger another retrieval attempt;** relevant ones proceed to the LLM.
- **LLM Processing:** LLM processes the query with relevant information and produces an initial answer.
- **Validation Process:** Performs a hallucination check to ensure the answer aligns with the context.
- **Self-Reflection:**
 - Critiques the generated response.
 - If unsatisfactory, triggers a query rewrite and restarts the process.
- **Final Output:** Once an accurate answer is generated, it is presented as the final output.

Speculative Agentic RAG :

Definition: Speculative RAG is a framework that enhances Retrieval-Augmented Generation (RAG) by using a larger generalist language model (LM) to verify multiple drafts produced in parallel by a smaller, distilled specialist LM.

Key Features:

- **Parallel Drafting:** Generates multiple answer drafts from distinct subsets of retrieved documents, providing diverse perspectives while reducing input token counts.
- **Efficiency:** The specialist LM handles drafting, allowing the generalist LM to perform a single verification pass, which accelerates the overall process.
- **Position Bias Mitigation:** By clustering retrieved documents based on content similarity, it minimizes redundancy and maximizes diversity in the drafting process.

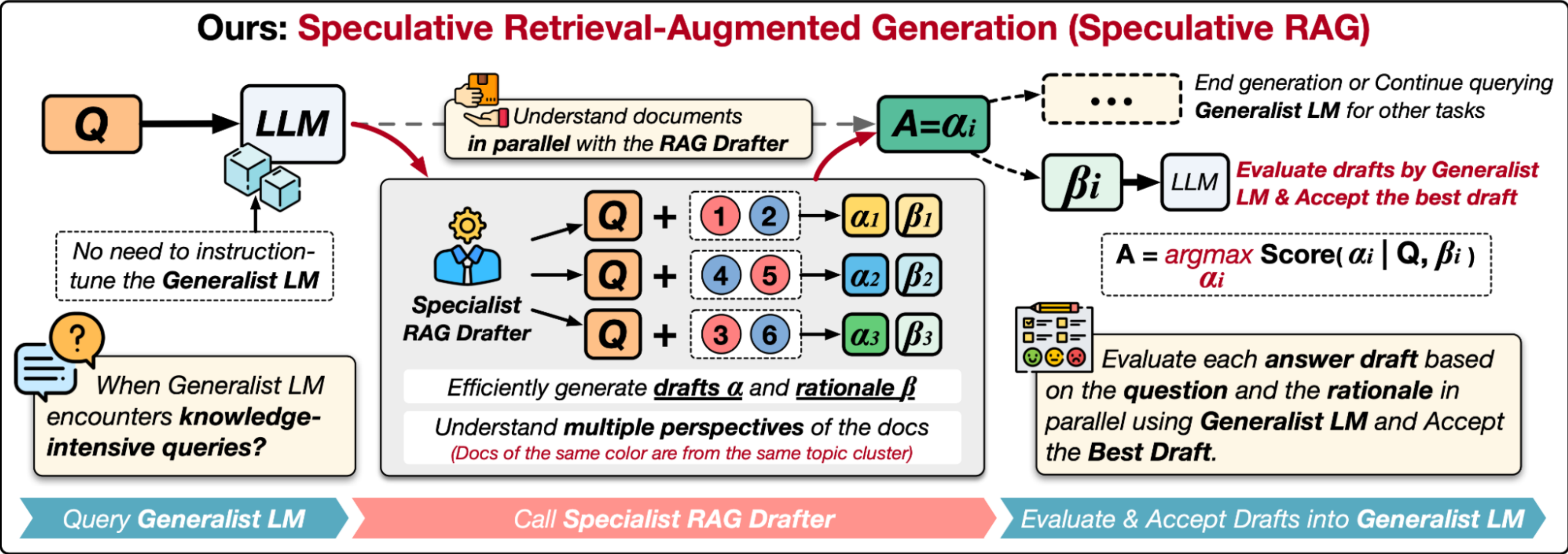
Performance Improvements: Achieves state-of-the-art results with accuracy enhancements of up to 12.97% and latency reductions of 51% compared to conventional RAG systems on various benchmarks like TriviaQA and PubHealth.

Applications: Effective for tasks requiring accurate and contextually rich responses while maintaining processing speed.

This framework addresses challenges in traditional RAG systems by efficiently managing computational resources and improving the response quality.

Speculative Agentic RAG :

Q A Knowledge-intensive Query **1 2** Retrieved Documents **A** Evidence-supported Answers **LLM** A Generalist LM  A Specialist LM



Self Route Agentic RAG :

Definition: An advanced implementation of Agentic RAG that emphasizes autonomous decision-making and dynamic query routing for optimized information retrieval and response generation.

Key Features:

- **Dynamic Query Routing:** Autonomously determines the best path for information retrieval based on user queries.
- **Multi-Agent Collaboration:** Utilizes specialized agents to handle complex tasks efficiently.
- **Real-Time Decision Making:** Agents assess the need for additional information and select appropriate tools based on context.

Process Steps:

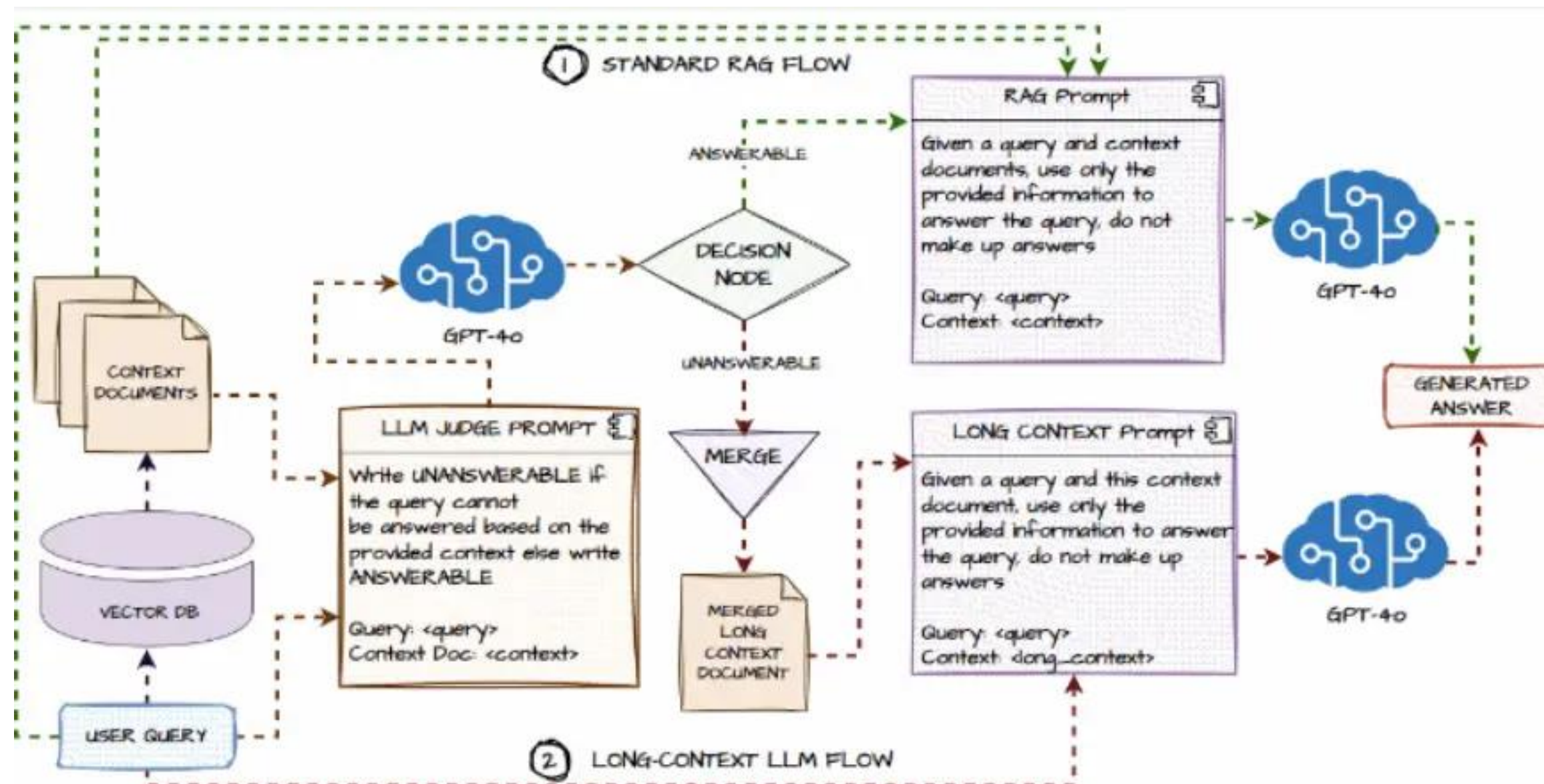
- **User Query Submission:** User inputs a query.
- **Routing Decision:** Agent decides if more retrieval is needed and selects the knowledge source.
- **Information Retrieval:** Retrieves relevant data from chosen sources.
- **Response Generation:** Processes the information to generate a coherent answer.
- **Self-Reflection and Adjustment:** Evaluates response quality; rewrites the query or retrieves more data if necessary.

Applications: Effective in customer support, healthcare diagnostics, and legal advisory services, adapting to user needs and improving response quality.

Self Route Agentic RAG :

Key components of Self Route:

1. **Decision-making by LLMs:** Queries are evaluated to determine if they can be answered with the given context.
2. **Routing:** If a query is answerable, it is processed immediately. Otherwise, it is routed to a long-context model with additional or full context.
3. **Efficiency and Accuracy:** This design balances **cost-efficiency** (avoiding unnecessary computation) and **accuracy** (leveraging long-context models only when needed).



Traditional RAG VS Agentic RAG :

Aspect	Traditional RAG	Agentic RAG
Interaction Model	Static queries to retrieve information	Dynamic interactions with retrieval and action steps
Adaptability	Limited to pre-defined queries and responses	Adapts actions based on real-time context and feedback
Control Flow	Linear and predefined	Non-linear and flexible with conditional logic
Autonomy	Minimal autonomy, reliant on user inputs	High autonomy with decision-making capabilities
Complexity of Tasks	Handles straightforward retrieval tasks	Manages complex, multi-step tasks involving multiple systems
User Involvement	Requires frequent user guidance	Reduces user involvement through proactive actions
Error Handling	Limited error handling and recovery mechanisms	Robust error handling with self-correction capabilities



WAS THIS POST USEFUL?

**FOLLOW FOR
MORE!**

